Machado User Manual

machado provides users with a framework to store, search, and visualize biological data.

It is powered by Django and contains tools to interact with a Chado database.

Contents

1. Installation
2. Data Loading
   - Ontologies
   - Taxonomy
   - Organism
   - Publication
   - FASTA
   - GFF
   - Feature Additional Info
   - BLAST
   - InterProScan
   - OrthoMCL
   - RNA-seq
   - Coexpression
   - VCF
3. Visualization
   - Index and Search
   - Web Server
   - JBrowse
   - Cache
4. Diagrams
5. Models

Technical Stack Documentation

For developer-oriented documentation covering the database schema, REST API, loaders internals, and CI/CD, see the techstack/ directory.

1. Installation

machado is a Django framework for Chado.

Prerequisites

PostgreSQL 16

Install PostgreSQL and create a database and user for loading the Chado schema. As the `postgres` user run:

```
psql
CREATE USER username WITH ENCRYPTED PASSWORD 'password';
CREATE DATABASE yourdatabase WITH OWNER username;
ALTER USER username CREATEDB;
```

Don't forget to configure the PostgreSQL server to allow regular users to connect (`pg_hba.conf`).

Linux dependencies

Be sure to have the following packages installed:

```
sudo apt install zlib1g-dev libbz2-dev liblzma-dev python3-dev
```

Python 3.12+

We strongly recommend creating a new virtualenv for your project:

```
mkdir /var/www/MYGENOME
cd /var/www/MYGENOME
python3 -m venv .venv
source .venv/bin/activate
```

Install machado

```
pip install machado
```

Or, to install from the latest source:

```
pip install git+https://github.com/lmb-embrapa/machado.git
```

Create a project

The `machado-startproject` command creates a ready-to-use Django project with pre-configured settings, URL routing, and WSGI/ASGI entry points:

```
machado-startproject .
```

This generates the following files in the current directory:

```
.
|-- .env                # Your configuration (auto-generated SECRET_KEY)
|-- .env.example        # Reference with all available settings
|-- manage.py
+-- machadoproject/
    |-- __init__.py
    |-- settings.py
    |-- urls.py
    |-- wsgi.py
    +-- asgi.py
```

Configure the environment

Edit the `.env` file and set your PostgreSQL connection string:

```
SECRET_KEY=<auto-generated>
DATABASE_URL=postgres://username:password@localhost:5432/yourdatabase
```

The `DATABASE_URL` format follows the dj-database-url convention. Replace `username`, `password`, and `yourdatabase` with the values you created in the Prerequisites step.

See `.env.example` for the full list of optional settings including Elasticsearch, JBrowse, and API configuration.

Migrate and run

Apply the Chado schema migrations:

```
python manage.py migrate
```

Just ignore warnings about unapplied migrations.

Run the tests to verify the installation:

```
python manage.py test machado
```

Start the development server:

```
python manage.py runserver
```

Open `http://localhost:8000/` in your browser.

References

- http://gmod.org/wiki/Chado_Django_HOWTO

2. Data Loading

The Chado schema heavily relies on **Ontologies** to integrate different datasets. Therefore ontologies must be the first data loaded.

Inside the `extras` directory, there's a file named `sample.tar.gz` that might be helpful. It contains a few small FASTA and GFF files, together with a README file with examples of command lines.

Loading order

The recommended loading order is:

1. Ontologies — **required**, must be loaded first
2. Taxonomy — optional, but useful for multi-organism instances
3. Organism — required if taxonomy was not loaded
4. Publication — optional, allows linking DOIs to data
5. FASTA — reference genome sequences
6. GFF — gene models / genome annotations
7. Feature Additional Info — annotations, sequences, publications, cross-references
8. BLAST — similarity search results
9. InterProScan — protein domain analysis
10. OrthoMCL — orthology groups
11. RNA-seq — expression data
12. Coexpression — coexpression pairs and clusters
13. VCF — variant data

> **Tip:** Most `load_*` commands accept `--cpu` to increase the number of threads and speed up loading.

Loading Ontologies

The ontologies are required and must be loaded first.

Relations Ontology

RO is an ontology of relations for use with biological ontologies.

- **URL:** https://github.com/oborel/obo-relations
- **File:** `ro.obo`

```
python manage.py load_relations_ontology --file ro.obo
```

Sequence Ontology

Collection of SO Ontologies.

- **URL:** https://github.com/The-Sequence-Ontology/SO-Ontologies
- **File:** `so.obo`

```
python manage.py load_sequence_ontology --file so.obo
```

Gene Ontology

Source ontology files for the Gene Ontology.

- **URL:** http://current.geneontology.org/ontology/
- **File:** `go.obo`

```
python manage.py load_gene_ontology --file go.obo
```

- Loading the Gene Ontology can be faster if you increase the number of threads (`--cpu`).
- After loading, the following records will be created in the Cv table: `gene_ontology`, `external`, `molecular_function`, `cellular_component`, and `biological_process`.

Remove Ontology

If, for any reason, you need to remove an ontology, use the command `remove_ontology`. Most data files you'll load depend on the ontologies (e.g. FASTA, GFF, BLAST). You should **never** delete an ontology after having data files loaded.

```
python manage.py remove_ontology --help
```

- This command requires the name of the ontology (`Cv.name`).
- There are dependencies between the Gene Ontology records, therefore you need to delete the entry `external` first.

Loading Taxonomy

Every data file you'll load (e.g. FASTA, GFF, BLAST) must belong to an organism. These are instructions to load the NCBI taxonomy, that contains most organisms you'll need.

This step is optional. If you decide to skip this step, you should insert the organisms individually using the command `insert_organism`.

NCBI Taxonomy

Contains the names and phylogenetic lineages of more than 160,000 organisms that have molecular data in the NCBI databases.

- **URL:** https://www.ncbi.nlm.nih.gov/taxonomy
- **File:** ftp://ftp.ncbi.nlm.nih.gov/pub/taxonomy/taxdump.tar.gz

After unpacking the file you'll have the required files.

```
python manage.py load_organism --file names.dmp --name DB:NCBI_taxonomy
python manage.py load_phylotree --file nodes.dmp --name 'NCBI taxonomy tree' --organismdb 'DB:NCBI_taxo
```

- Loading these files can be faster if you increase the number of threads (`--cpu`).
- It will take a long time anyway (hours).

Remove Taxonomy

If, for any reason, you need to remove a taxonomy, use the commands `remove_phylotree` and `remove_organisms`. Most data files you'll load depend on the organism database (e.g. FASTA, GFF, BLAST). **If you delete an organism database, every data file you loaded will be deleted on cascade.**

```
python manage.py remove_phylotree --help
python manage.py remove_organisms --help
```

- These commands require the names of the databases (`Db.name`).

Inserting a New Organism

Insert Organism

Every data file you'll load (e.g. FASTA, GFF, BLAST) must belong to an organism. These are instructions to insert a new organism to the database **if you did NOT** load the NCBI taxonomy or if the organism you're working with is not included.

```
python manage.py insert_organism --genus 'Organismus' --species 'ineditus'
```

- There are optional fields you can provide. Please take a look at the command help (`--help`).

Remove Organism

If, for any reason, you need to remove an organism, use the command `remove_organism`. Most data files you'll load depend on the organism record (e.g. FASTA, GFF, BLAST). **If you delete an organism, every data file you loaded that depends on it will be deleted on cascade.**

```
python manage.py remove_organism --help
```

- These commands require the following info: `Organism.genus` and `Organism.species`.

Loading Publication Files

Load Publication

```
python manage.py load_publication --file reference.bib
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

Remove Publication

If, for any reason, you need to remove a publication, use the command `remove_publication`. **If you delete a publication, every record that depends on it will be deleted on cascade, with the exception of the Dbxref field that contains the DOI accession.**

```
python manage.py remove_publication --help
```

Loading FASTA Files

This command is mostly used to load the reference genome. The reference sequences are exclusively used to feed JBrowse.

If the reference sequences are really long (>200 Mbp), there may be memory issues during the loading process and JBrowse may take too long to render the tracks. To avoid that, it's possible to use the parameter `--nosequence` and configure JBrowse to get the reference data from a FASTA file.

Load FASTA

```
python manage.py load_fasta --file organism_chrs.fa --soterm chromosome --organism 'Arabidopsis thaliana
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_fasta --help
```

Parameter	Description
`--file` *	FASTA file
`--organism` *	Species name (e.g. *Homo sapiens*, *Mus musculus*)
`--soterm` *	SO Sequence Ontology Term (e.g. chromosome, assembly)
`--description`	Description
`--url`	URL
`--doi`	DOI of a reference stored using `load_publication` (e.g. 10.1111/s12122-012-1313-4)
`--nosequence`	Don't load the sequences
`--cpu`	Number of threads

* required fields

Remove File

If, for any reason, you need to remove a FASTA file, use the command `remove_organism`. Most data files you'll load depend on the organism record (e.g. FASTA, GFF, BLAST). **If you delete an organism, every data file you loaded that depends on it will be deleted on cascade.**

```
python manage.py remove_organism --help
```

- These commands require the following info: `Organism.genus` and `Organism.species`.

Loading GFF Files

The first column of a GFF file is the reference sequence ID. Usually, in order to load a GFF file, it's required to have a reference FASTA file loaded. But some GFF files already have the reference features such as chromosome or scaffold. In this case, there are two options:

- Load the GFF directly, without a reference FASTA file
- Load the FASTA file and then load the GFF using the parameter `--ignore` to not load the reference features

The GFF file must be indexed using tabix.

Load GFF

```
python manage.py load_gff --file organism_genes_sorted.gff3.gz --organism 'Arabidopsis thaliana'
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_gff --help
```

Parameter	Description
`--file` *	GFF3 genome file indexed with tabix (see http://www.htslib.org/doc/tabix.html)
`--organism` *	Species name (e.g. *Homo sapiens*, *Mus musculus*)
`--ignore`	List of feature types to ignore (e.g. chromosome scaffold)
`--doi`	DOI of a reference stored using `load_publication` (e.g. 10.1111/s12122-012-1313-4)
`--qtl`	Set this flag to handle GFF files from QTLDB

Parameter	Description
`--cpu`	Number of threads

* required fields

Remove File

If, for any reason, you need to remove a GFF dataset, use the command `remove_file`. **If you delete a file, every record that depends on it will be deleted on cascade.**

```
python manage.py remove_file --help
```

- This command requires the file name (`Dbxrefprop.value`).

Loading Feature Additional Info

Please notice the features must be loaded in advance. If the annotation or sequence was loaded previously, it will be replaced.

Load Annotation

```
python manage.py load_feature_annotation --file feature_annotation.tab --soterm polypeptide --cvterm di
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_feature_annotation --help
```

Parameter	Description
`--file` *	Two-column tab separated file (`feature.accession<TAB>annotation text`)
`--organism` *	Species name (e.g. *Homo sapiens*, *Mus musculus*)
`--soterm` *	SO Sequence Ontology Term (e.g. mRNA, polypeptide)
`--cvterm` *	cvterm.name from cv feature_property (e.g. display, note, product, alias, ontology_term, annotation)
`--doi`	DOI of a reference stored using `load_publication` (e.g. 10.1111/s12122-012-1313-4)
`--cpu`	Number of threads

* required fields

Remove Annotation

If, for any reason, you need to remove a feature annotation, use the command `remove_feature_annotation`.

```
python manage.py remove_feature_annotation --help
```

Load Sequence

```
python manage.py load_feature_sequence --organism 'Arabidopsis thaliana' --file Athaliana_transcripts.fa
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_feature_sequence --help
```

Parameter	Description
`--file` *	FASTA file
`--organism` *	Species name (e.g. *Homo sapiens*, *Mus musculus*)
`--soterm` *	SO Sequence Ontology Term (e.g. chromosome, assembly, mRNA, polypeptide)
`--cpu`	Number of threads

* required fields

Remove Sequence

If, for any reason, you need to remove a feature sequence, use the command `load_feature_sequence` itself and provide a FASTA file with no sequence. For example:

```
>chr1

>chr2
```

Load Publication

```
python manage.py load_feature_publication --organism 'Arabidopsis thaliana' --file feature_publication.t
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_feature_publication --help
```

Parameter	Description
`--file` *	Two-column tab separated file (`feature.accession<TAB>DOI`)
`--organism` *	Species name (e.g. *Homo sapiens*, *Mus musculus*)
`--cpu`	Number of threads

* required fields

Remove Publication

If, for any reason, you need to remove a feature publication attribution, use the command `remove_publication`.

```
python manage.py remove_publication --help
```

Load DBxRef

```
python manage.py load_feature_dbxrefs --organism 'Arabidopsis thaliana' --file feature_dbxrefs.tab --so
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_feature_dbxrefs --help
```

Parameter	Description
`--file` *	Two-column tab separated file (`feature.accession<TAB>db:dbxref`)
`--organism` *	Species name (e.g. *Homo sapiens*, *Mus musculus*)
`--soterm` *	SO Sequence Ontology Term (e.g. chromosome, assembly, mRNA, polypeptide)

Parameter	Description
`--cpu`	Number of threads

* required fields

Loading BLAST Results

In order to load a BLAST XML result file, both the query and the subject records must be previously stored (see below). The current version was tested for loading BLAST analysis on **proteins**.

Load BLAST Subject Records

In case you did a BLAST against a multispecies protein database, like NCBI's nr or Uniprot's trembl or swissprot, you need to load all subject matches before loading the result itself:

```
python manage.py load_similarity_matches --file blast_result.xml --format blast-xml
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

Load BLAST

If all queries and subjects are already loaded you can run the following command to load a BLAST XML result:

```
python manage.py load_similarity --file blast_result.xml --format blast-xml \
    --so_query polypeptide --so_subject protein_match \
    --program diamond --programversion 0.9.24 \
    --organism_query 'Oryza sativa' --organism_subject 'multispecies multispecies'
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_similarity --help
```

Parameter	Description
`--file` *	BLAST XML file
`--format` *	`blast-xml`
`--so_query` *	Query Sequence Ontology term (e.g. assembly, mRNA, polypeptide)
`--so_subject` *	Subject Sequence Ontology term (e.g. protein_match)
`--organism_query` *	Query's organism name (e.g. *Oryza sativa*). Cannot be 'multispecies'.
`--organism_subject` *	Subject's organism name. Put `multispecies multispecies` if using a multispecies database.
`--program` *	Program
`--programversion` *	Program version
`--name`	Name
`--description`	Description
`--algorithm`	Algorithm
`--cpu`	Number of threads

* required fields

Remove File

If, for any reason, you need to remove a BLAST result set, use the command `remove_analysis`.

```
python manage.py remove_analysis --help
```

- This command requires the analysis name (`Analysis.sourcename`).

Loading InterProScan Results

In order to load an InterProScan results file, both the query and the subject records must be previously stored. The current version was tested for loading InterProScan analysis on **proteins**.

Load InterProScan Subject Records

```
python manage.py load_similarity_matches --file interproscan_result.xml --format interproscan-xml
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

Load InterProScan Similarity

```
python manage.py load_similarity --file interproscan_result.xml --format interproscan-xml \
    --so_query polypeptide --so_subject protein_match \
    --program interproscan --programversion 5 \
    --organism_query 'Oryza sativa' --organism_subject 'multispecies multispecies'
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_similarity --help
```

Parameter	Description
`--file` *	InterProScan XML file
`--format` *	`interproscan-xml`
`--so_query` *	Query Sequence Ontology term (`polypeptide`)
`--so_subject` *	Subject Sequence Ontology term (`protein_match`)
`--organism_query` *	Query's organism name (e.g. *Oryza sativa*). Cannot be 'multispecies'.
`--organism_subject` *	Subject's organism name. Put `multispecies multispecies` if using a multispecies database.
`--program` *	Program
`--programversion` *	Program version
`--name`	Name
`--description`	Description
`--algorithm`	Algorithm
`--cpu`	Number of threads

* required fields

Remove File

If, for any reason, you need to remove an InterProScan result set, use the command `remove_analysis`.

```
python manage.py remove_analysis --help
```

- This command requires the analysis name (`Analysis.sourcename`).

Loading OrthoMCL Results

Load OrthoMCL

```
python manage.py load_orthomcl --file groups.txt
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_orthomcl --help
```

Parameter	Description
`--file` *	Output result from OrthoMCL software
`--cpu`	Number of threads

* required fields

Remove Orthology

If, for any reason, you need to remove orthology relationships, use the command `remove_feature_annotation`.

```
python manage.py remove_feature_annotation --help
```

Parameter	Description
`--cvterm` *	`orthologous group`
`--organism`	Species name (e.g. *Homo sapiens*, *Mus musculus*)

* required fields

Loading RNA-seq Data

Load RNA-seq Information

Before inserting RNA-seq count tables, it is needed to input information about the experiments and samples from which the data was generated. In machado we focus on the GEO/NCBI database as a source for RNA-seq experiments information and data.

From the GEO/NCBI database you get identifiers for different *series* (e.g. GSE85653) that describe the studies/projects as a whole. From the GEO series you get identifiers for biosamples (or, in Chado lingo, biomaterials — e.g. GSM2280286). From the GEO biosamples you get identifiers for RNA-seq experiments (or assays), usually from the SRA database (e.g. SRR4033018). SRA identifiers link to the raw data you can analyse.

In machado, it is necessary to input a `.csv` file with information for all SRA data files regarding RNA-seq assays that will be loaded.

This file must have the following fields in each line:

```
"Organism", "GEO series", "GEO sample", "SRA identifier", "Assay description", "Treatment description",
```

A sample line:

```
Oryza sativa,GSE85653,GSM2280286,SRR4033018,Heat leaf rep1,Heat stress,Leaf,May-30-2018
```

To load such a file:

```
python manage.py load_rnaseq_info --file file.csv --biomaterialdb GEO --assaydb SRA
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_rnaseq_info --help
```

Parameter	Description
`--file` *	`.csv` file
`--biomaterialdb`	Biomaterial database info (e.g. `GEO`)
`--assaydb`	Assay database info (e.g. `SRA`)
`--cpu`	Number of threads

* required field

Remove RNA-seq Information

If, for any reason, you need to remove RNA-seq information, use the command `remove_file --name`. **Every relation from the filename (e.g. `file.csv`) will be deleted on cascade.**

```
python manage.py remove_file --help
```

- This command requires the file name used as input to load RNA-seq information.

Load RNA-seq Data

To load expression count tables for RNA-seq data, a tabular file should be loaded. This file can contain data from several RNA-seq experiments (assays) per column. The file should have the following header:

```
Gene identifier    SRA Identifier 1    SRA Identifier 2    ...    SRA Identifier n
```

Example header for a file with two assays/experiments:

```
gene    SRR5167848.htseq    SRR2302912.htseq
```

The body of the table is composed of the gene identifier followed by the counts for each gene, in each experiment:

```
AT2G44195.1.TAIR10     0.0     0.6936967934559419
```

Note that the count fields can have floats or integers, depending on the normalization used (usually TPM, FPKM or raw counts).

The gene identifier is supposed to already be loaded as a feature, usually from the organism's genome annotation GFF file. We used the output of the LSTrAP program as standard format for this file.

```
python manage.py load_rnaseq_data --file file.tab --organism 'Oryza sativa' --programversion 1.3 --assay
```

- As default the program name is `LSTrAP` but can be changed with `--program`.
- The data is by default taken as normalized (TPM, FPKM, etc.) but can be changed with `--norm`.
- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_rnaseq_data --help
```

Parameter	Description
`--file`	Tabular text file with gene counts per line
`--organism`	Scientific name (e.g. *Oryza sativa*)
`--programversion`	Version of the software (e.g. `1.3`)
`--name`	Optional name
`--description`	Optional description

Parameter	Description
`--algorithm`	Optional algorithm description
`--assaydb`	Optional assay database info (e.g. `SRA`)
`--timeexecuted`	Optional date software was run. Mandatory format: e.g. `Oct-16-2016`
`--program`	Optional name of the software (default: `LSTrAP`)
`--norm`	Optional. Normalized data: `1` = yes (TPM, FPKM, etc.); `0` = no (raw counts); default `1`

Remove RNA-seq Data

If, for any reason, you need to remove RNA-seq data, use the command `remove_file --name`. **Every relation from the filename (e.g. `file.tab`) will be deleted on cascade.**

```
python manage.py remove_file --help
```

- This command requires the file name used as input to load RNA-seq data.

Loading Coexpression Analyses

Load Coexpression Correlated Pairs

Coexpression analyses usually generate correlation statistics regarding gene sets in a pairwise manner. For example, LSTrAP can generate a table with the Pearson correlation coefficient for every pair of genes in an RNA-seq experiment.

To load such a table into machado, the file must be headless and tab separated, with the two first columns containing the correlated pair of IDs for the genes/features and the third column containing the correlation coefficient. In the case of the LSTrAP software output, the coefficient is subtracted by 0.7 for normalization.

A sample from such a table:

```
AT2G44195.1.TAIR10      AT1G30080.1.TAIR10      0.18189286870895194
AT2G44195.1.TAIR10      AT5G24750.1.TAIR10      0.1715779378273995
```

> **Note:** The feature pairs from columns 1 and 2 need to be loaded previously.

To load such a table:

```
python manage.py load_coexpression_pairs --organism 'Arabidopsis thaliana' --file pcc.mcl.txt
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_coexpression_pairs --help
```

Parameter	Description
`--file` *	Coexpression pairs file (e.g. `pcc.mcl.txt`)
`--organism` *	Species name (e.g. *Homo sapiens*, *Mus musculus*)
`--soterm`	Sequence ontology term (default: `mRNA`)
`--cpu`	Number of threads

* example output file from LSTrAP software

Remove Coexpression Pairs

If, for any reason, you need to remove coexpression pair analyses, pass the filename used to load the analyses to the `remove_relationship` command. **Every coexpression relation from the file will be deleted on cascade.**

```
python manage.py remove_relationship --help
```

Parameter	Description
`--file` *	Coexpression file (e.g. `mcl.clusters.txt`)

Load Coexpression Clusters

Another type of coexpression analysis involves clustering features using their correlation values. LSTrAP does this using the `mcl` software. The input file must be headless and tab separated, with each line representing one cluster and each column representing one gene/feature from that cluster. The first column should represent the cluster name in the format `<cluster_name>:`.

Three-cluster sample (the third line is an orphaned cluster with only one individual — orphaned clusters are discarded):

```
ath_1:    AT3G18715.1.TAIR10      AT3G08790.1.TAIR10      AT5G42230.1.TAIR10
ath_2:    AT1G27040.1.TAIR10      AT1G71692.1.TAIR10
ath_3:    AT5G24750.1.TAIR10
```

> **Note:** The genes/features from each column must be loaded previously.

To load such a file:

```
python manage.py load_coexpression_clusters --file mcl.clusters.txt --organism 'Arabidopsis thaliana'
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_coexpression_clusters --help
```

Parameter	Description
`--file` *	Clusters file (e.g. `mcl.clusters.txt`)
`--organism`	Scientific name (e.g. *Arabidopsis thaliana*)
`--soterm`	Sequence ontology term (default: `mRNA`)
`--cpu`	Number of threads

Remove Coexpression Clusters

If, for any reason, you need to remove coexpression cluster analyses, pass the controlled vocabulary term `coexpression group` and the organism scientific name to the command `remove_feature_annotation`. **Every coexpression group relation from that organism will be deleted on cascade.**

```
python manage.py remove_feature_annotation --help
```

Parameter	Description
`--cvterm` *	`coexpression group`
`--organism`	Scientific name (e.g. *Oryza sativa*)

Loading VCF Files

The first column of a VCF file is the reference sequence ID. In order to load a VCF file, it's required to have a reference FASTA file loaded.

The VCF file must be indexed using tabix.

Load VCF

```
python manage.py load_vcf --file organism_snv_sorted.vcf.gz --organism 'Arabidopsis thaliana'
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_vcf --help
```

Parameter	Description
`--file` *	VCF genome file indexed with tabix (see http://www.htslib.org/doc/tabix.html)
`--organism` *	Species name (e.g. *Homo sapiens*, *Mus musculus*)
`--doi`	DOI of a reference stored using `load_publication` (e.g. 10.1111/s12122-012-1313-4)
`--cpu`	Number of threads

* required fields

Remove File

If, for any reason, you need to remove a VCF dataset, use the command `remove_file`. **If you delete a file, every record that depends on it will be deleted on cascade.**

```
python manage.py remove_file --help
```

- This command requires the file name (`Dbxrefprop.value`).

Visualization

The machado web interface requires the installation and configuration of third party software.

- Index and Search — PostgreSQL full-text search
- Web Server — Development server and Apache WSGI deployment
- JBrowse — Genome browser integration (optional)
- Cache — Caching configuration (optional)

Index and Search

PostgreSQL Full-Text Search

Machado uses the native **full-text search capabilities of PostgreSQL** to provide fast, reliable search and faceted navigation without requiring any external search services. All search indexes are stored within the main database.

```
MACHADO_VALID_TYPES=gene,mRNA,polypeptide
```

If `MACHADO_VALID_TYPES` is not explicitly set in the settings, the default is `['gene', 'mRNA', 'polypeptide']`.

Indexing the Data

The search system relies on a denormalized "materialized" index table to guarantee fast responses. After loading data into the Chado database, you must build the search index to make the new records searchable:

```
python manage.py rebuild_search_index
```

> **Note:** It is necessary to run `rebuild_search_index` whenever additional data is loaded into the database or when you wish to refresh the search facets.

Rebuilding the index queries the underlying database tables and batch-inserts the records into the search index. You can control the batch size to tune performance:

```
# Process 5,000 records at a time
python manage.py rebuild_search_index --batch-size 5000
```

Search Features

The PostgreSQL search backend supports advanced search queries natively using "websearch" syntax:

- `gene kinase` — searches for documents containing both words
- `"receptor kinase"` — searches for the exact phrase
- `kinase -receptor` — searches for "kinase" but excludes documents containing "receptor"
- `kinase OR receptor` — searches for either word

All exported data (TSV or FASTA) from the search page is unlimited; PostgreSQL handles exporting the full result set regardless of its size.

Web Server

Django manage runserver

Check if your server is working — test the machado server:

```
python manage.py runserver
```

Now, open your browser and go to http://127.0.0.1:8000.

Use `CTRL+C` to stop the web server.

Django Apache WSGI

Before starting, make sure you have Apache installed in your system. In Ubuntu 20.04:

```
sudo apt install apache2
```

In order to have Apache2 hosting the Django applications, it's necessary to use WSGI. By doing this it won't be necessary to run the runserver command anymore — Apache will take care of this process. Install the following package:

```
sudo apt install libapache2-mod-wsgi-py3
```

Now configure Apache to use the WSGI module. Here is the configuration file (`/etc/apache2/sites-available/MYGENOME.con`

```
<Directory "/var/www/MYGENOME/machadoproject">
    <Files "wsgi.py">
        Require all granted
    </Files>
</Directory>

Alias /MYGENOME/static/ /var/www/MYGENOME/staticfiles/

<Directory "/var/www/MYGENOME/staticfiles">
    Require all granted
</Directory>

WSGIDaemonProcess machadoproject python-home=/var/www/MYGENOME/.venv python-path=/var/www/MYGENOME lang=
WSGIProcessGroup machadoproject
WSGIScriptAlias /MYGENOME /var/www/MYGENOME/machadoproject/wsgi.py
```

- In this example the whole project is in `/var/www/MYGENOME`, but it's not required to be there.
- This directory and sub-directories must have 755 permissions.
- Ensure the `python-home` path matches your virtual environment (`.venv`).

There must be a symlink of your config file in the sites-enabled directory:

```
sudo ln -s /etc/apache2/sites-available/MYGENOME.conf /etc/apache2/sites-enabled/MYGENOME.conf
```

In the `.env` file, ensure the following variables are set for production:

```
DEBUG=False
ALLOWED_HOSTS=*

MACHADO_VALID_TYPES=gene,mRNA,polypeptide
```

Now, run `collectstatic` to gather the static files from all libraries to the `staticfiles` directory:

```
python manage.py collectstatic
```

It's necessary to restart the Apache2 service every time there are modifications on configuration files or source code updates:

```
sudo systemctl restart apache2.service
```

Now, open your browser and go to your configured domain (e.g., http://yourdomain.com/MYGENOME).

JBrowse (optional)

The JBrowse software renders genomic tracks from Chado using the machado API.

Installing JBrowse

Before installing JBrowse you should probably have BioPerl installed in your system (tested in Ubuntu 20.04):

```
sudo apt install bioperl
```

Then install JBrowse following the official instructions.

In Ubuntu 20.04, install some prerequisites:

```
sudo apt install build-essential zlib1g-dev curl
```

Also Node.js is needed:

```
curl -sL https://deb.nodesource.com/setup_12.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Finally, proceed with JBrowse installation:

```
wget https://github.com/GMOD/jbrowse/releases/download/1.16.12-release/JBrowse-1.16.12.zip
unzip JBrowse-1.16.12.zip
sudo mv JBrowse-1.16.12 /var/www/html/jbrowse
cd /var/www/html
sudo chown `whoami` jbrowse
cd jbrowse
./setup.sh  # don't do sudo ./setup.sh
```

Configuring JBrowse

The machado source contains the directory `extras`. Copy the file `extras/trackList.json.sample` to the JBrowse data directory, inside a subdirectory with the name of the organism (e.g. `/var/www/html/jbrowse/data/Arabidopsis`

thaliana) and rename it to trackList.json. This directory structure will enable machado to embed JBrowse in its pages.

```
mkdir -p "/var/www/html/jbrowse/data/Arabidopsis thaliana"
cp extras/trackList.json.sample "/var/www/html/jbrowse/data/Arabidopsis thaliana/trackList.json"
```

Edit the file trackList.json to set the **organism** name you have loaded to the database.

In case you have WSGI apache module configured and running, change the baseUrl variable in trackList.json to refer to the proper address:

```
"baseUrl": "http://localhost/MYGENOME/api/jbrowse"
```

- Repeat the steps above for as many other organisms as you may have loaded to the database.
- Remember to restart the Apache server after the modifications.

machado API

In case you don't have the WSGI module installed under Apache (and did not change the baseUrl variable in trackList.json), start the machado API framework:

```
python manage.py runserver
```

Once the server is running, go to your browser and open your JBrowse instance (e.g. http://localhost/jbrowse/?data=data/

machado Settings

The settings.py file should contain these variables in order to have the JBrowse visualization embedded in the feature page. Don't forget to restart Apache to make the settings changes up to date.

```
MACHADO_JBROWSE_URL = 'http://localhost/jbrowse'
MACHADO_JBROWSE_TRACKS = 'ref_seq,gene,transcripts,CDS,SNV'
MACHADO_JBROWSE_OFFSET = 1200
```

- MACHADO_JBROWSE_URL — the base URL to the JBrowse installation. The URL must contain the protocol (i.e. http or https).
- MACHADO_JBROWSE_TRACKS — the name of the tracks to be displayed (ref_seq,gene,transcripts,CDS,SNV if not set).
- MACHADO_JBROWSE_OFFSET — the number of bp upstream and downstream of the feature (1000 if not set).

Use Reference from FASTA File (optional)

If the reference sequences are really long (>200 Mbp), there may be memory issues during the loading process and JBrowse may take too long to render the tracks. To avoid this, follow instructions to create and use an indexed FASTA file as source for the reference sequences — see the JBrowse tutorial.

Follow the steps below.

Put the genome's assembly FASTA file into your JBrowse organism's data/seq directory (for example: /var/www/html/jbrowse/data/Glycine max/data/seq/Gmax.fa), change to this directory, and run:

```
samtools faidx Gmax.fa
```

A Gmax.fa.fai indexed FASTA file will be created.

Now, modify your default trackList.json file. You need to make two modifications — first replace the refSeqs entry line (probably the second line of the file) with:

```
"refSeqs" : "data/seq/Gmax.fa.fai",
```

And then change the reference sequence track code chunk:

```
{
    "category" : "1. Reference sequence",
    "faiUrlTemplate" : "data/seq/Gmax.fa.fai",
    "key" : "Reference sequence",
    "label" : "ref_seq",
    "seqType" : "dna",
    "storeClass" : "JBrowse/Store/SeqFeature/IndexedFasta",
    "type" : "SequenceTrack",
    "urlTemplate" : "data/seq/Gmax.fa",
    "useAsRefSeqStore" : 1
}
```

- The code above should replace all code from the "1. Reference sequence" category track code chunk.
- Make sure `urlTemplate` points to the FASTA file path, not the `.fai` indexed one.

Now restart the Apache daemon for changes to take effect:

```
systemctl restart apache2
```

Cache (optional)

The machado views are pre-configured to enable local-memory caching for 1 hour. In order to move the cache to another location, you'll need to set Django's cache framework following the official instructions.

Example

Include the following in `settings.py` to store the cache in the database:

```
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.db.DatabaseCache',
        'LOCATION': 'machado_cache_table',
    }
}
CACHE_TIMEOUT = 60 * 60 * 24  # 86400 seconds == 1 day
```

Create the cache table:

```
python manage.py createcachetable
```

Clearing the Cache Table

It is a good idea to clear the cache table whenever you make changes to your machado installation. For this, install the django-clear-cache tool:

```
pip install django-clear-cache
```

Then modify your `settings.py` file to add `clear_cache`:

```
INSTALLED_APPS = (
    # ...
    'clear_cache',
)
```

Then to clear the cache just run:

```
python manage.py clear_cache
```

Diagrams

AnalysisFeature

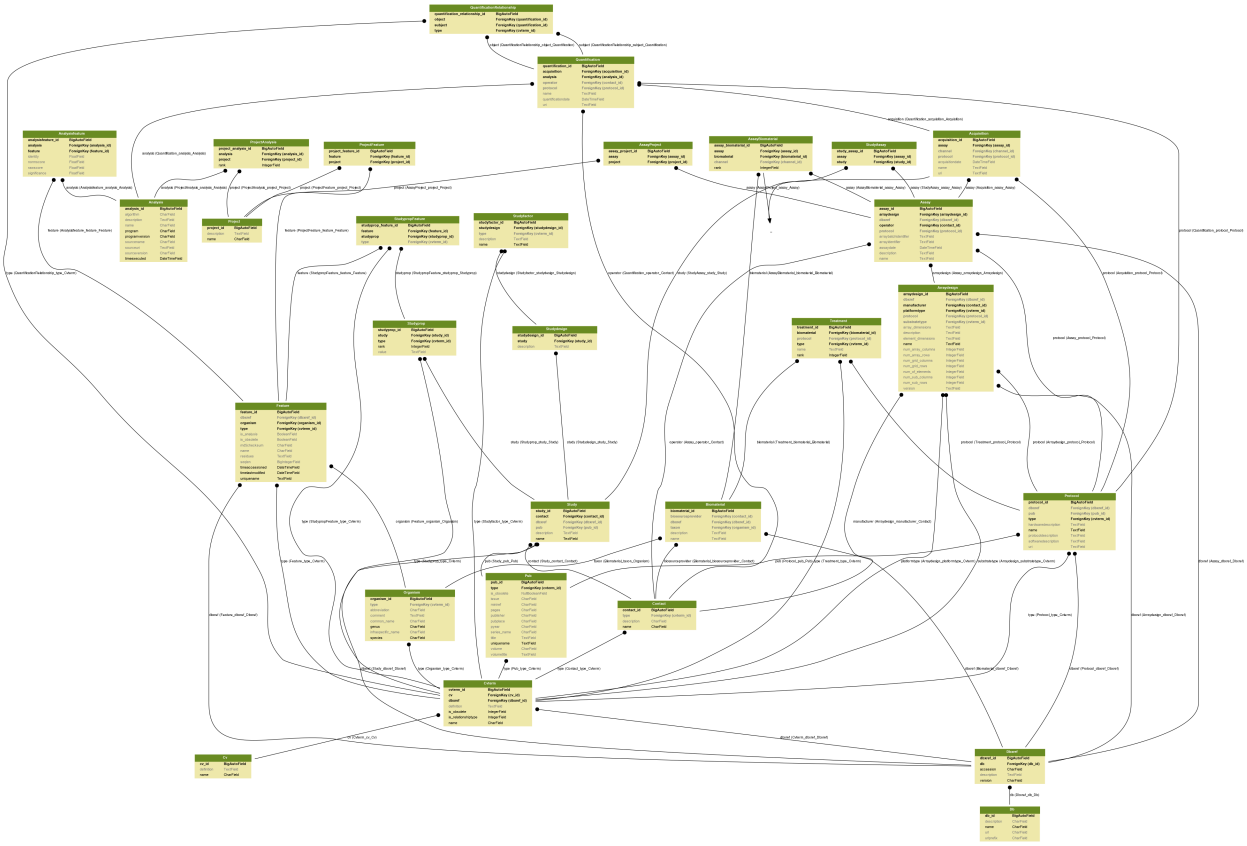

Figure 1: AnalysisFeature diagram

Feature

Ontology

Publication

Sequence

Similarity

Taxonomy

Note: Instructions to generate the diagrams: https://django-extensions.readthedocs.io/en/latest/graph_models.html

Example: `python manage.py graph_models --pygraphviz -a -I Analysis,Analysisfeature,Feature -o analysis_feature.png`

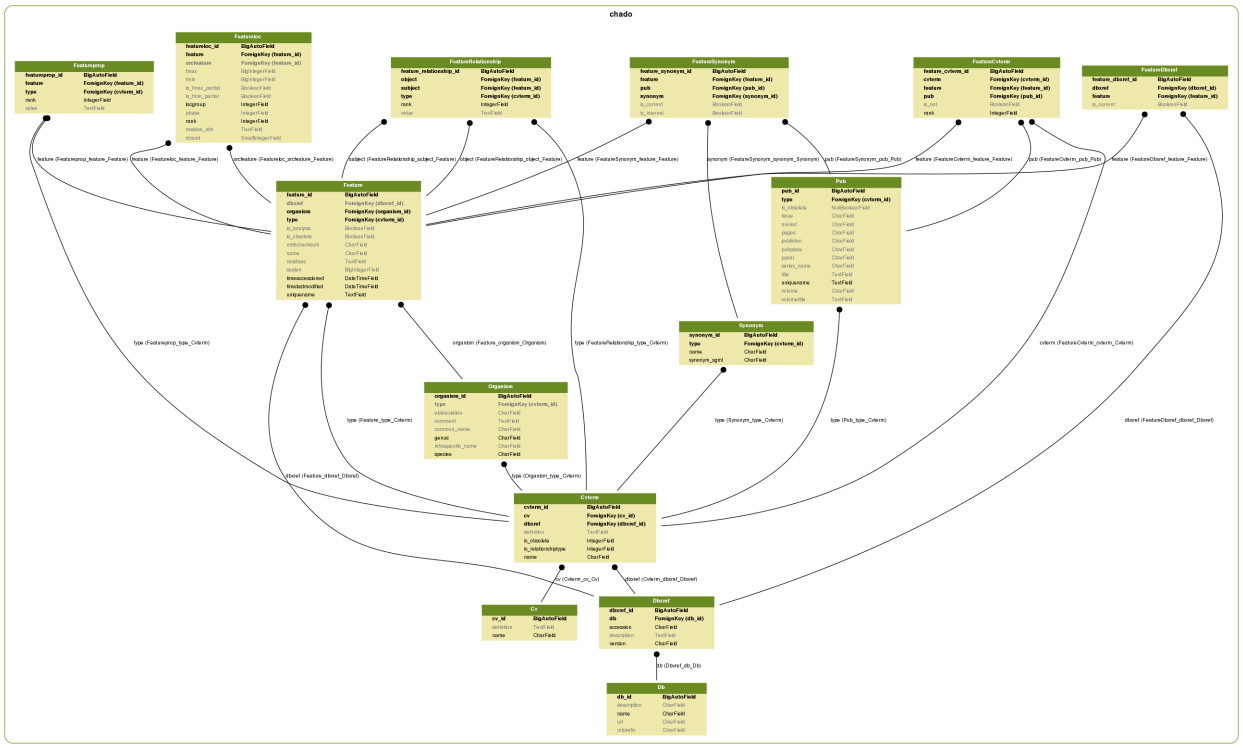

Figure 2: Feature diagram

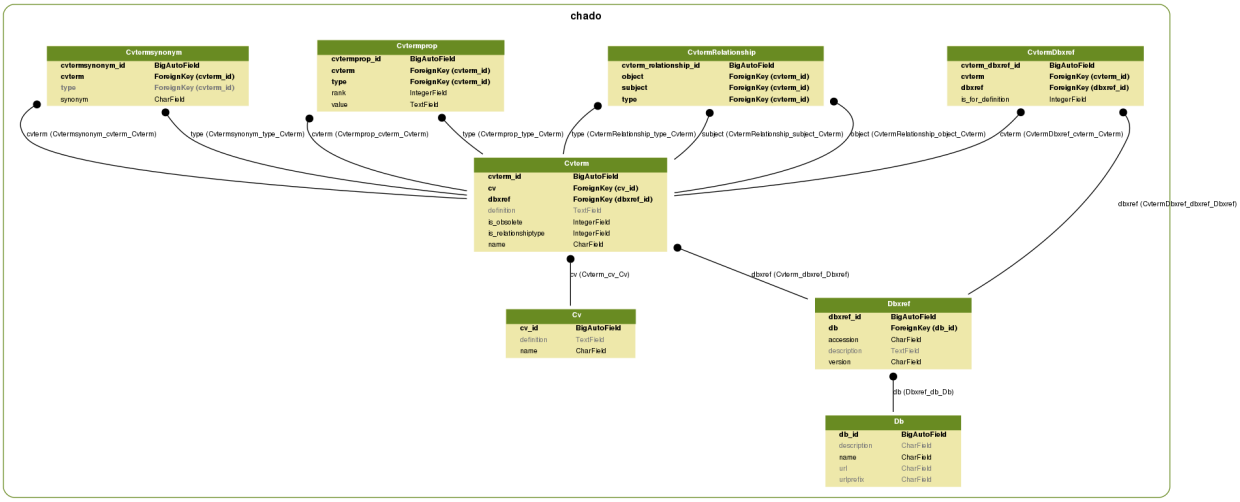

Figure 3: Ontology diagram

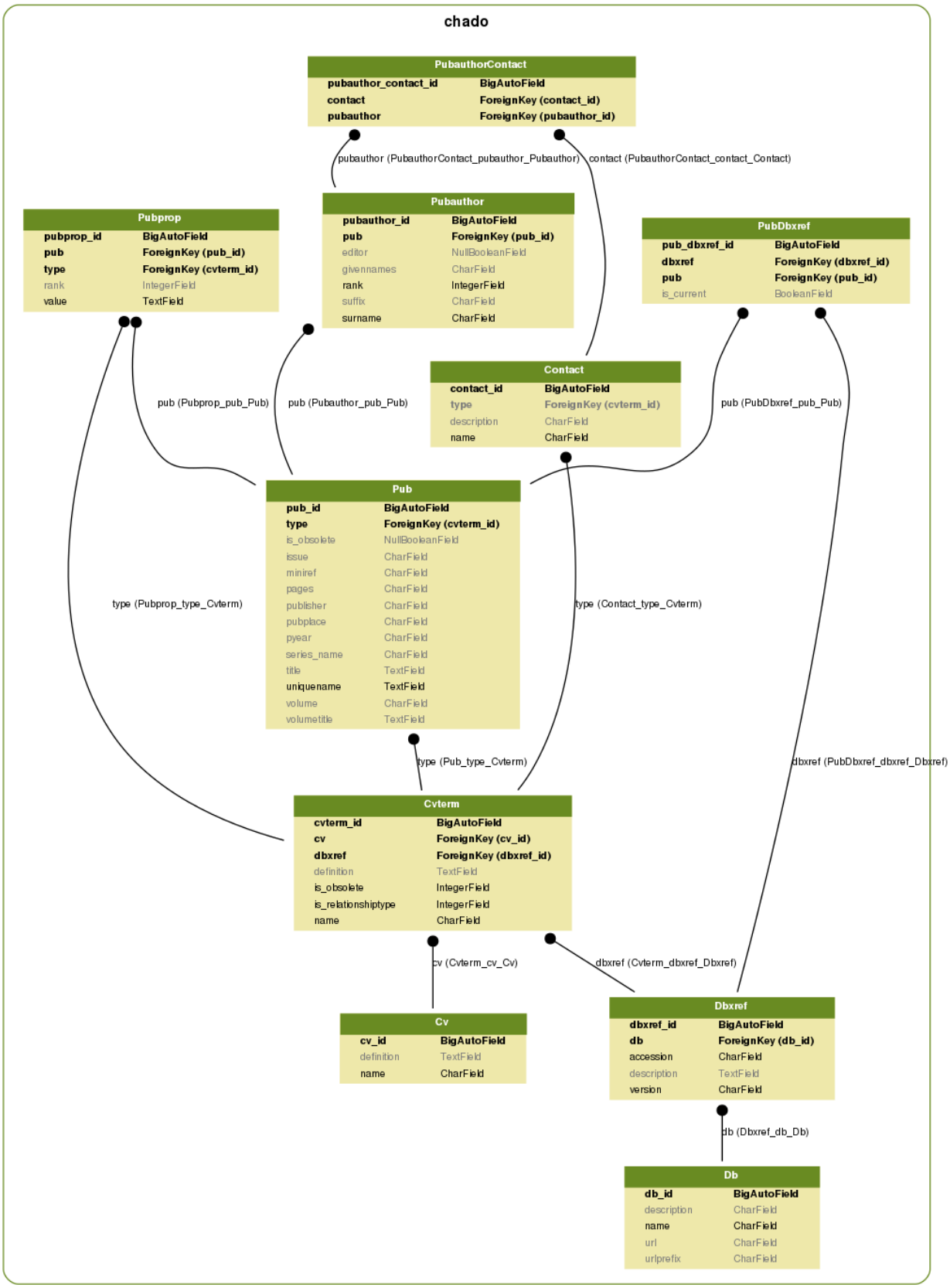

Figure 4: Publication diagram

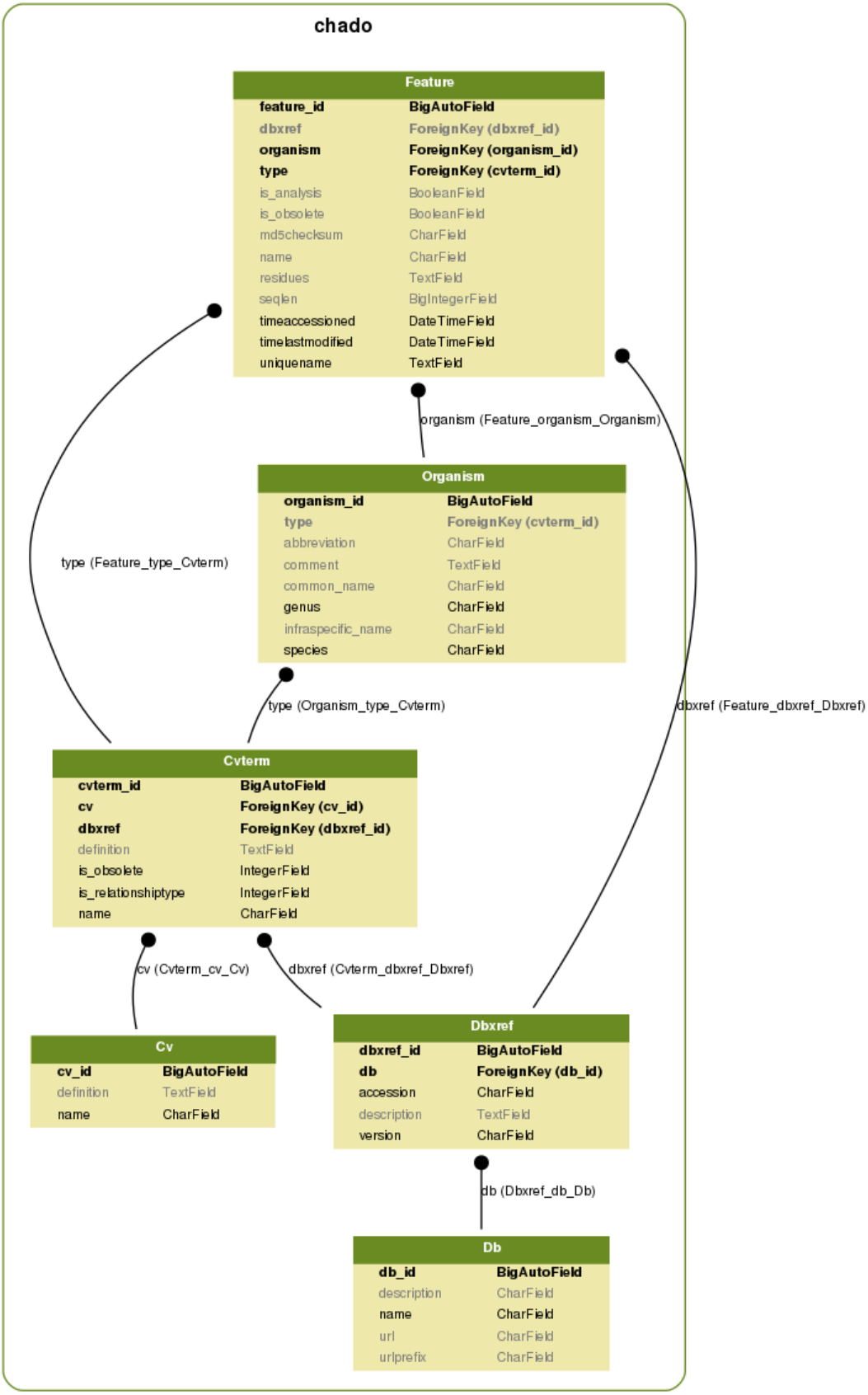

Figure 5: Sequence diagram

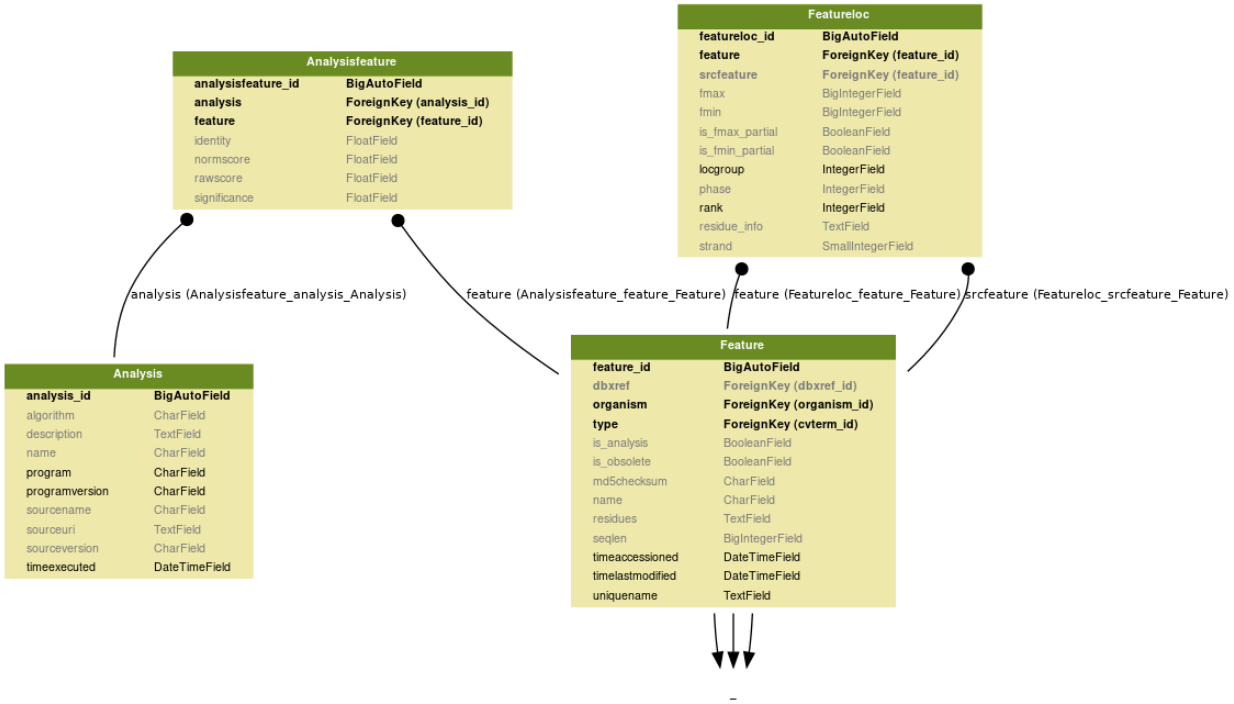

Figure 6: Similarity diagram

Models

This document describes how the Django `models.py` file for the Chado schema was created. You **don't need** to create it again since machado already contains a copy of this file.

Prerequisite

The list below contains the software and versions required by machado.

PostgreSQL 9.5

Install PostgreSQL and create a database and user for loading the Chado schema. As the `postgres` user run:

```
psql
CREATE USER username WITH ENCRYPTED PASSWORD 'password';
CREATE DATABASE yourdatabase WITH OWNER username;
```

Chado 1.31

Download the Chado schema, unpack the file and load the `chado-1.31/schemas/1.31/default_schema.sql` to the database:

```
psql -h localhost -U username -W -d yourdatabase < chado-1.31/schemas/1.31/default_schema.sql
```

Python 3.5.2

We strongly recommend creating a new virtualenv for your project:

```
virtualenv -p /usr/bin/python3 MYGENOME
cd MYGENOME
```

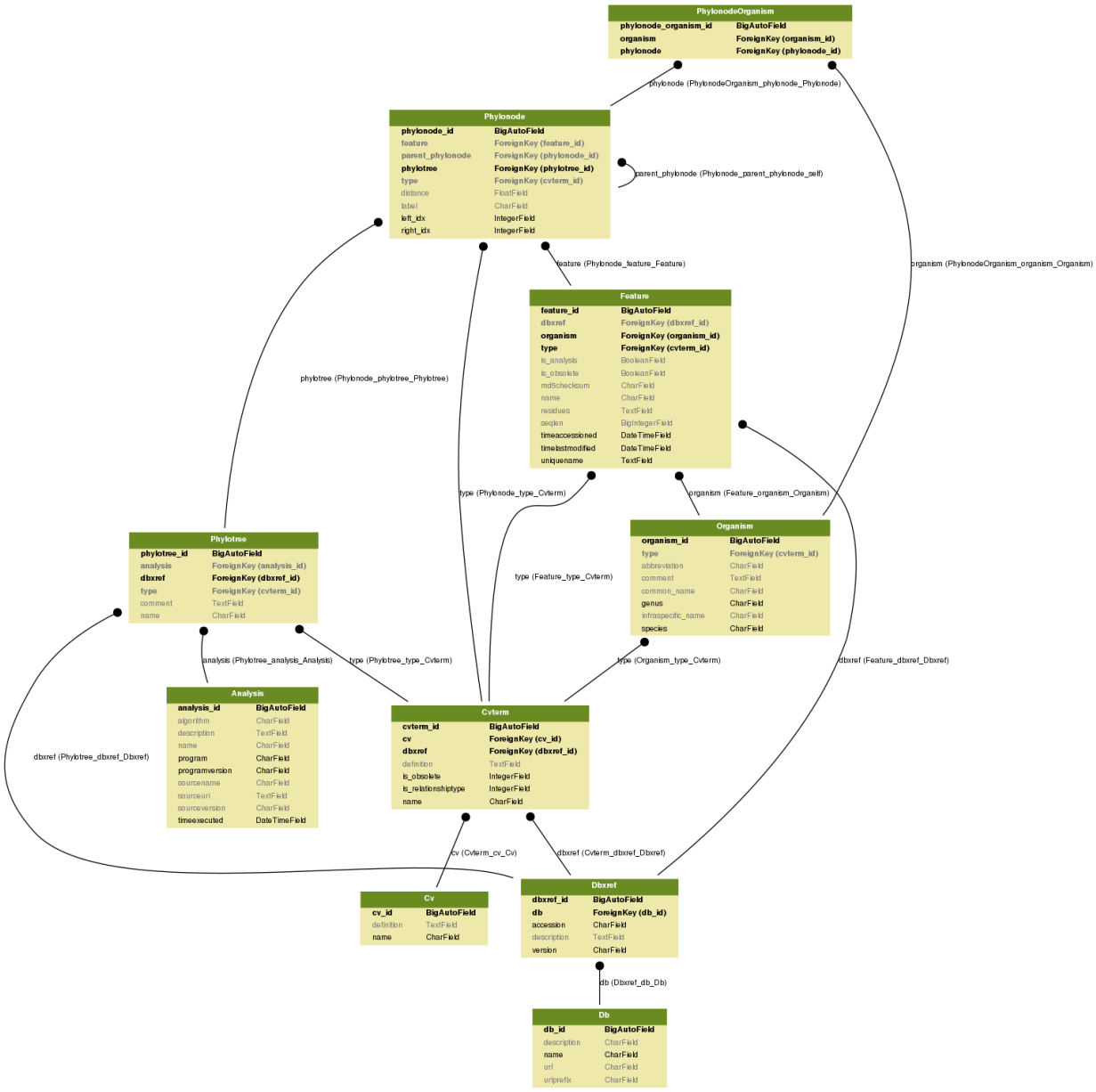

Figure 7: Taxonomy diagram

```
source bin/activate
```

machado

Just grab the code using Git and install it:

```
git clone https://github.com/lmb-embrapa/machado.git src/machado
python src/machado/setup.py install
```

The Django Project

Inside the `MYGENOME` directory, create a Django project with the following command:

```
django-admin startproject WEBPROJECT
cd WEBPROJECT
```

Then, configure the `WEBPROJECT/settings.py` file to connect to your Chado database:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql_psycopg2',
        'NAME': 'yourdatabase',
        'USER': 'username',
        'PASSWORD': 'password',
        'HOST': 'localhost',
        'PORT': '',
    },
}
```

The Model

Django has a command to generate a models file:

```
python manage.py inspectdb > unsortedmodels.py
```

This will create a raw `models.py` with a model for each table and view in the specified Postgres database. This file needs to be fixed as each foreign key relation should have a unique name in Django to support reverse relationships. The following Python code will create these unique names. The code rewrites the models and also generates an `admin.py` file:

```
fixChadoModel.py --input unsortedmodels.py
```

The resulting files, `models.py` and `admin.py`, are ready.

References

- http://gmod.org/wiki/Chado_Django_HOWTO